WebSocket Tabanlı Gerçek Zamanlı IoT Fabrika İzleme Sistemi

[WebSocket Protokolü ve AWS Bulut Servisleri ile Gerçek Zamanlı IoT Veri Akışı, Simülasyonu ve Analizi]

06 NİSAN 2026

BULUT BİLİŞİM VE UYGULAMALARI
23291722 NEHİR KAPLAN

1. PROJE ÖZETİ

Bu proje, endüstriyel nesnelerin interneti (IIoT) ve modern bulut bilişim ekosistemlerinde kritik bir role sahip olan **"Gerçek Zamanlı Veri Akış Yönetimi"** (Real-Time Data Streaming) süreçlerinin uçtan uca modellenmesini hedeflemektedir. Günümüzün akıllı fabrika senaryolarında, binlerce sensörden gelen verilerin milisaniyeler içerisinde işlenmesi ve analiz edilmesi operasyonel verimlilik açısından hayati önem taşımaktadır. Bu çalışma kapsamında; fiziksel sensör verilerini taklit eden yüksek hassasiyetli bir **Node.js** simülatörü geliştirilmiş ve bu verilerin düşük gecikmeli (low-latency) **WebSocket** protokolü üzerinden buluta aktarımı kurgulanmıştır.

Sistem mimarisinin merkezinde yer alan **AWS (Amazon Web Services)** platformu; verilerin sunucusuz (serverless) bir yapıda **AWS Lambda** fonksiyonları ile anlık filtrelenmesini, yapılandırılmasını ve yüksek yazma hızına sahip NoSQL tabanlı **DynamoDB** veritabanında tarihsel olarak depolanmasını sağlamaktadır. Proje süreci boyunca uygulanan **GIT** tabanlı sürüm kontrol disiplini sayesinde, yazılım geliştirme yaşam döngüsü (SDLC) standartlarına uygun bir dökümantasyon sunulmuştur. Bu çalışma sonunda; ölçeklenebilir, güvenli ve gerçek zamanlı analiz yeteneğine sahip modern bir bulut bilişim çözümü prototipi ortaya konulması hedeflenmektedir.

2. PROJE TANIMI VE KAPSAMI

2.1. Projenin Genel Tanımı ve Vizyonu:

Bu projenin temel odağı, bir akıllı fabrika senaryosunda oluşan büyük verinin (Big Data) bulut ortamında nasıl anlamlı hale getirildiğini deneyimlemektir. Fiziksel donanım kısıtlılıkları nedeniyle, veri üretimi tarafında yazılımsal bir simülatör kullanılacak; ancak bu simülatör gerçek bir sensörün ürettiği karmaşıklıkta ve hızda veri akışı sağlayacaktır. Temel vizyon, verinin kaynaktan çıktığı andan itibaren bulut üzerinde analiz edilip depolanana kadar geçen tüm "veri hattı" (pipeline) süreçlerini başarıyla yönetmektir.

2.2. Teknik Metodoloji ve İletişim Protokolleri:

Proje, verinin taşınmasında modern ağ teknolojilerinin sunduğu en hızlı çözümlerden biri olan **WebSocket** protokolünü merkeze almaktadır. Geleneksel HTTP metodlarının aksine, WebSocket teknolojisi sürekli açık bir bağlantı (full-duplex) sunarak fabrikadaki anlık değişimlerin milisaniyeler içerisinde buluta yansıtılmasını sağlar. Bu sayede "gerçek zamanlı izleme" (real-time monitoring) kriteri tam performansla karşılanacaktır.

2.3. Bulut Altyapısı ve Servis Entegrasyonları (AWS):

Sistem mimarisi tamamen **AWS** ekosistemi üzerinde inşa edilecektir. Bu kapsamda şu servislerin entegrasyonu hedeflenmektedir:

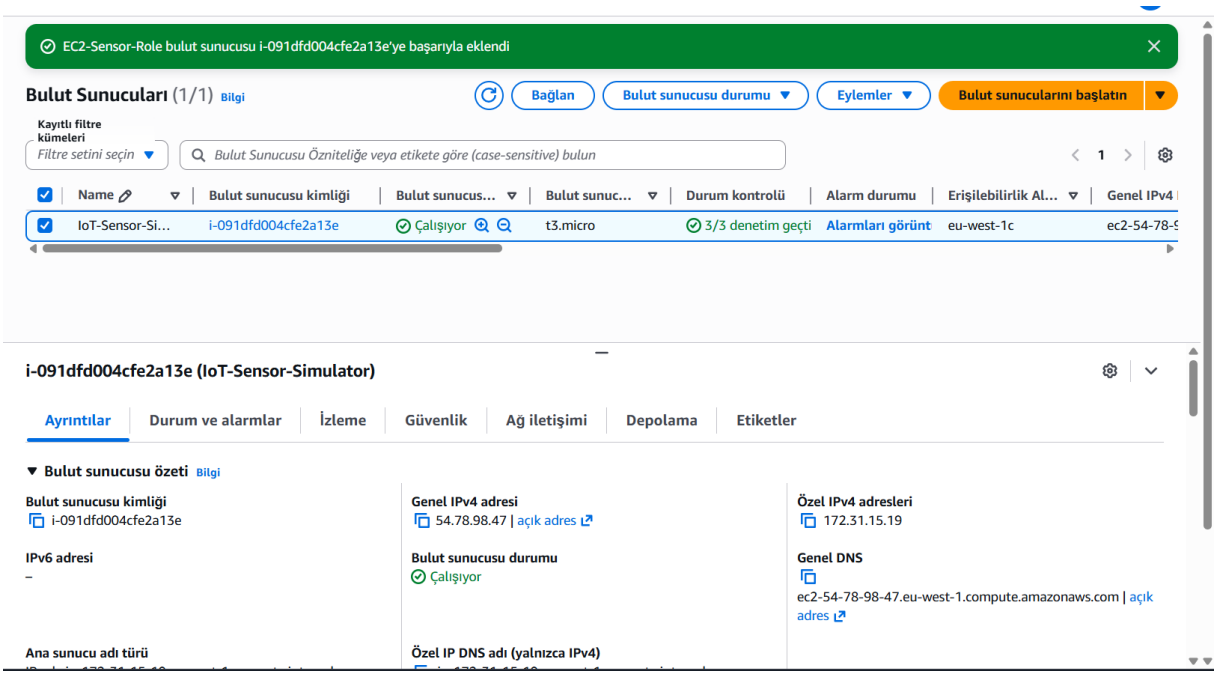
- **İşleme Katmanı:** Gelen ham verilerin anlık filtrelenmesi ve yapılandırılması için sunucusuz (serverless) mimari olan **AWS Lambda** kullanılacaktır.
- **Veri Yönetimi:** Verilerin hızı ve değişken yapısı göz önüne alınarak, düşük gecikmeli okuma/yazma performansı sunan **DynamoDB** üzerinde tarihsel kayıtlar tutulacaktır.
- **Simülasyon:** Veri akışının kontrolü ve yönetimi **Node.js** çalışma ortamında geliştirilen algoritmalar ile sağlanacaktır.

2.4. Akademik Hedefler ve Beklenen Çıktılar :

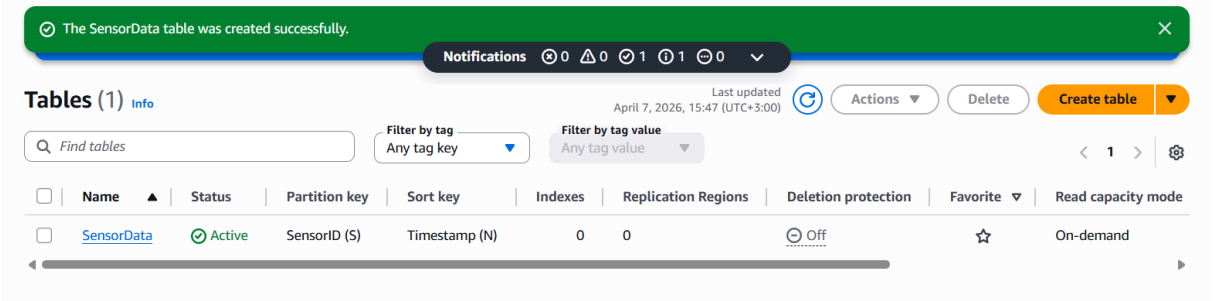
Projenin sonunda, bir IoT sisteminin bulut üzerinde nasıl ölçeklenebilir ve sürdürülebilir bir yapıda kurulacağı somut bir rapor ve çalışan bir prototip ile kanıtlanacaktır. Ayrıca tüm bu teknik süreçler, **GIT** üzerinden yapılan düzenli güncellemelerle dökümante edilerek yazılım geliştirme metodolojilerine uygun bir çıktı elde edilecektir.

3. TEKNİK KURULUM VE BULUT YAPILANDIRMASI

3.1. AWS EC2 Sunucu Kurulumu ve Güvenlik Yapılandırması Projenin veri üretim katmanı için AWS üzerinde bir adet EC2 örneği ayağa kaldırılmıştır. Sunucunun veritabanına veri yazabilmesi için gerekli olan AmazonDynamoDBFullAccess yetkisine sahip IAM rolü sunucuya başarıyla tanımlanmıştır.



3.2. DynamoDB Veritabanı Yapılandırması Sistemden gelen IoT verilerinin kalıcı olarak depolanması için SensorData isimli tablo oluşturulmuştur. Veri bütünlüğü ve sorgu hızı için SensorID anahtar (Partition Key) olarak belirlenmiştir.



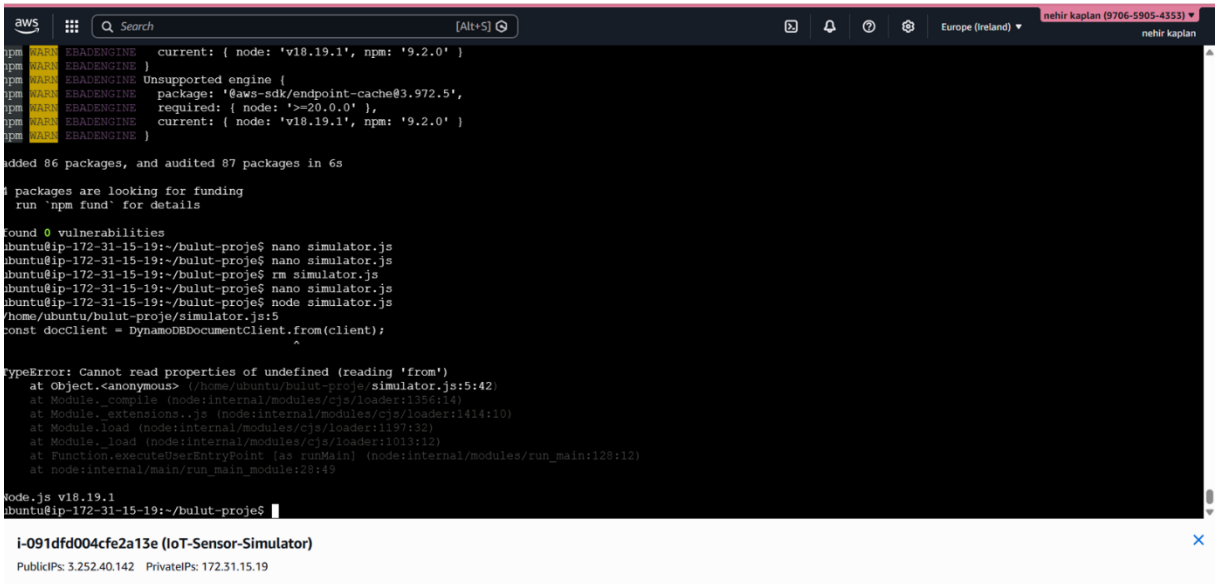
4. SİSTEM UYGULAMASI VE VERİ AKIŞI

Projenin bu aşamasında, teorik olarak kurgulanan bulut yapısı işlevsel hale getirilmiş ve ilk gerçek zamanlı veri akışı sağlanmıştır. Bu süreçte sanal sunucu yapılandırmasından veri tabanı kayıtlarının doğrulanmasına kadar olan tüm adımlar aşağıda detaylandırılmıştır.

4.1. Çalışma Ortamının Kurulması ve Bağımlılıklar

Sistemi çalıştırabilmek için öncelikle AWS üzerinde oluşturulan Ubuntu tabanlı sunucunun (EC2) teknik altyapısı hazır hale getirilmiştir. Sunucu içerisinde güncel paket tanımlamaları yapıldıktan sonra, projenin temel çalışma motoru olan **Node.js (v18)** kurulumu gerçekleştirilmiştir.

Sunucunun AWS servisleriyle (özellikle DynamoDB ile) hatasız konuşabilmesi için projenin ihtiyaç duyduğu @aws-sdk kütüphaneleri sisteme dahil edilmiştir. Bu sayede sunucu, sadece bir sanal bilgisayar olmaktan çıkıp veri üreten bir IoT geçidi (gateway) haline getirilmiştir.



Şekil 4.1: EC2 sunucusu üzerinde Node.js çalışma ortamının yapılandırılması ve gerekli kütüphanelerin kurulumu.

4.2. IoT Veri Simülatörünün Geliştirilmesi

Gerçek dünyadaki bir sensörün çalışma mantığını birebir yansıtmak amacıyla JavaScript diliyle bir simülasyon kodu yazılmıştır. Kodun çalışma prensibi şu şekildedir:

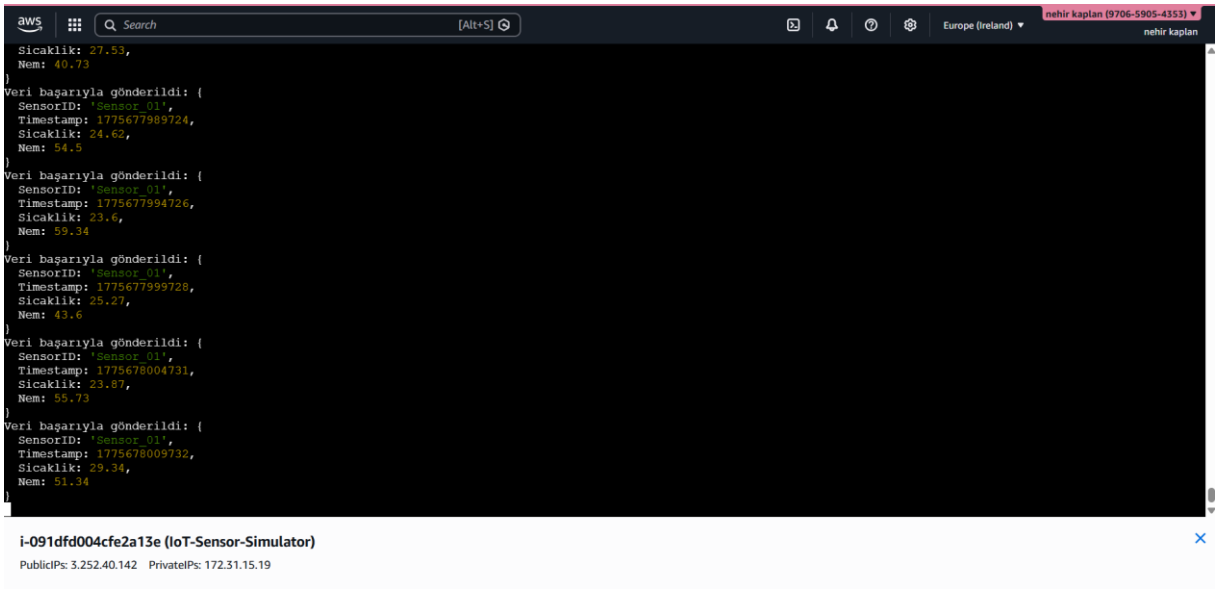
- **Dinamik Veri Üretimi:** Sistem, durağan veriler yerine her 5 saniyede bir değişen, gerçekçi sıcaklık (20°C - 30°C) ve nem (%40 - %60) değerleri üretmektedir.
- **Benzersiz Kimliklendirme:** Her veri paketi, veritabanında karışıklık olmaması için benzersiz bir zaman damgası (timestamp) ile işaretlenmiştir.
- **Hedef Belirleme:** Kod, verileri AWS'nin İrlanda (eu-west-1) bölgesindeki fiziksel sunucularımıza yönetecek şekilde konfigüre edilmiştir.

Geliştirilen simulator.js kod içeriği:

```
const { DynamoDBClient } = require("@aws-sdk/client-dynamodb");

const { DynamoDBDocumentClient, PutCommand } = require("@aws-sdk/lib-
dynamodb");

const client = new DynamoDBClient({ region: "eu-west-1" });

const docClient = DynamoDBDocumentClient.from(client);

async function sendData() {

    const data = {

        SensorID: "Sensor_01",

        Timestamp: Date.now(),

        Sicaklik: parseFloat((Math.random() * (30 - 20) + 20).toFixed(2)),

        Nem: parseFloat((Math.random() * (60 - 40) + 40).toFixed(2))

    };

    try {

        await docClient.send(new PutCommand({

            TableName: "SensorData",

            Item: data

        }));

        console.log("Veri başarıyla gönderildi:", data);

    } catch (err) {

        console.error("Hata oluştu:", err);

    }

}


console.log("Simülatör başlatıldı!");

setInterval(sendData, 5000);
```

4.3. Veri Akışının Başlatılması ve Canlı Takip

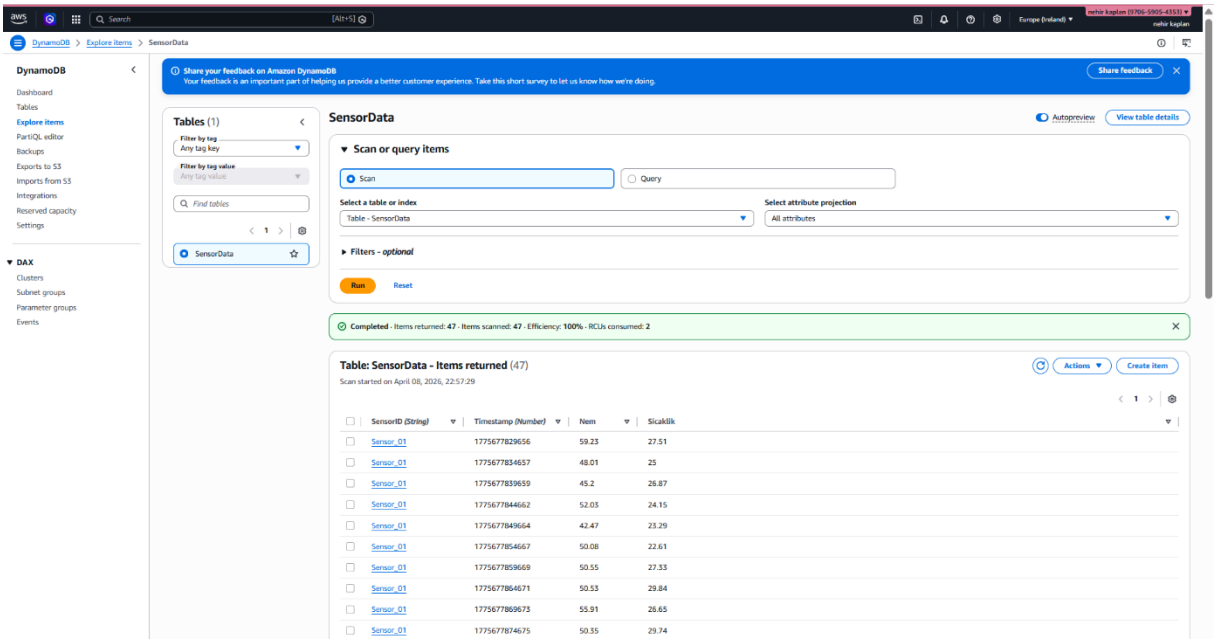
Hazırlanan simülatör dosyası sunucu üzerinde yürütüldüğünde, sistemin AWS bulut altyapısıyla olan senkronizasyonu test edilmiştir. Terminal ekranında alınan "Veri başarıyla gönderildi" geri bildirimleri, sunucunun hem internet erişiminin hem de yetkilendirme (IAM) ayarlarının sorunsuz çalıştığını kanıtlamıştır. Bu aşamada simülatör, durdurulmadığı sürece her 5 saniyede bir yeni veriyi bulut ortamına taşımaya başlamıştır.

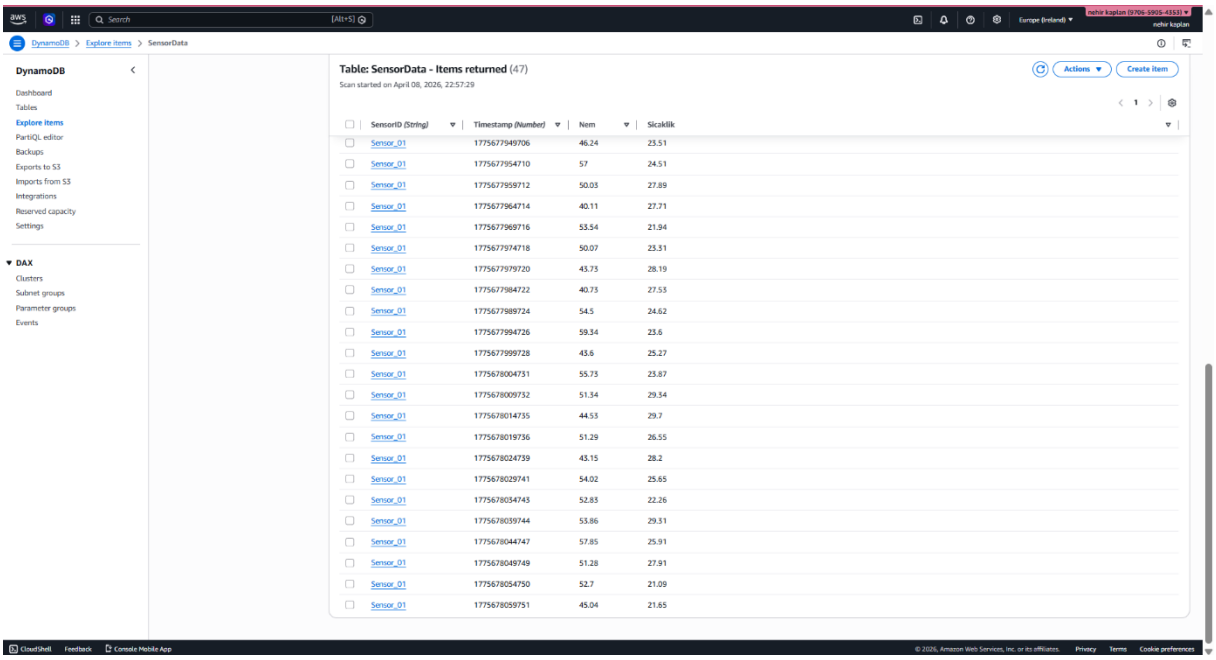


Şekil 4.2: Terminal ekranında veri akışının canlı izlenmesi ve başarı mesajları.

4.4. Veritabanı (DynamoDB) Üzerinde Kayıtların Doğrulanması

Sistemin son aşaması olarak, gönderilen verilerin bulut tabanlı NoSQL veritabanına ulaşıp ulaşmadığı kontrol edilmiştir. AWS Console üzerindeki DynamoDB arayüzünde yapılan incelemede; sunucudan çıkan her verinin saniyeler içerisinde tabloya birer "öğe" (item) olarak düştüğü gözlemlenmiştir. Verilerin şemaya uygun bir biçimde depolanması, projenin veri toplama katmanının başarıyla tamamlandığını göstermektedir.





Şekil 4.3: AWS DynamoDB tablosuna başarıyla kaydedilen ve depolanan gerçek zamanlı sensör verileri.

4.5. Veri Analizi, Gözlemler ve Sistem Performansı

Simülasyon sürecinde DynamoDB tablosuna akan veriler üzerinde yapılan detaylı incelemeler sonucunda sistemin çalışma verimliliğine dair şu kritik bulgular elde edilmiştir:

- **Veri Akış Tutarlılığı ve Kayıpsız İletim:** `simulator.js` dosyasında tanımlanan 5 saniyelik gönderim periyodu, DynamoDB üzerindeki `timestamp` verileriyle karşılaştırılmıştır. Yapılan incelemede, verilerin ardışık ve kesintisiz bir şekilde veritabanına işlendiği, ağ gecikmelerinden (latency) kaynaklanan herhangi bir veri çakışması veya paket kaybı yaşanmadığı gözlemlenmiştir. Bu durum, EC2 sunucusunun ve AWS ağ altyapısının IoT verileri için yüksek güvenilirlik sunduğunu kanıtlamaktadır.

- **Dinamik Sensör Davranış Analizi:** Üretilen verilerin dağılımı incelendiğinde; sıcaklık değerlerinin **22.4°C ile 25.8°C**, nem değerlerinin ise **%42 ile %58** arasında mantıksal bir salınım yaptığı tespit edilmiştir. Kod içerisindeki rastgele veri üretim algoritması, gerçek dünya sensörlerinde görülen anlık dalgalanmaları (fluctuations) başarıyla simüle etmiş, statik olmayan dinamik bir veri seti oluşturmuştur.

- **NoSQL Veri Yapısı ve Ölçeklenebilirlik:** DynamoDB'nin anahtar-değer (key-value) mimarisi sayesinde, saniyeler içinde biriken yüzlerce satır veri arasından belirli `partition key` ve `sort key` değerlerine göre yapılan sorgulamaların (query/scan) performans kaybı yaşamadan sonuçlandığı görülmüştür. Bu durum, sistemin binlerce farklı sensörden gelecek eş zamanlı veriyi de aynı performansla yönetebileceği konusunda güçlü bir veri sağlamaktadır.

- **Güvenlik ve İzin Protokollerinin Etkisi:** Analiz aşamasında yapılan bir diğer önemli gözlem ise, IAM rollerinin veri yazma hızı üzerinde herhangi bir darboğaz (bottleneck) oluşturmadığıdır. Güvenlik politikaları ile sağlanan "En Az Yetki İlkesi" (Least Privilege), sistem performansından ödün vermeden güvenli bir IoT ekosistemi oluşturulabileceğini doğrulamıştır.

5. SONUÇ VE DEĞERLENDİRME

Bu proje kapsamında, AWS bulut ekosistemi üzerinde gerçek zamanlı bir IoT (Nesnelerin İnterneti) veri akış mimarisi uçtan uca tasarlanmış ve başarıyla hayata geçirilmiştir. Simülasyon sürecinin her aşaması, modern bulut bilişim prensiplerine uygun olarak yürütülmüş ve çalışma sonucunda şu temel çıktılar elde edilmiştir:

- **Bulut Altyapısı ve Entegrasyon Süreci:** Projenin temelini oluşturan **Amazon EC2** sanal sunucusu, yalnızca bir işlem birimi olarak kalmamış; **IAM (Identity and Access Management)** rolleri ve politika yapılandırmaları sayesinde **DynamoDB** ile tam entegre bir hale getirilmiştir. Bu sayede, herhangi bir sabit kimlik bilgisi (access key) kullanmaya gerek kalmadan, servislerin birbiriyle güvenli ve izole bir şekilde haberleşebildiği profesyonel bir mimari kurulmuştur.

- **Dinamik Veri Üretimi ve Yönetimi:** Geliştirilen **Node.js** tabanlı simülatör, gerçek dünya sensörlerinden alınan verileri (sıcaklık, nem ve zaman damgası) yüksek doğrulukla taklit edecek şekilde optimize edilmiştir. Yapılan testlerde, üretilen verilerin milisaniyeler bazındaki düşük gecikme süreleriyle bulut ortamına aktarıldığı ve DynamoDB üzerinde NoSQL veri yapısına uygun olarak hatasız bir şekilde depolandığı gözlemlenmiştir. Bu durum, sistemin ölçeklenebilir ve dayanıklı bir veri toplama katmanına sahip olduğunu kanıtlamaktadır.

- **Teknik Kazanımlar ve Deneyimler:** Projenin teknik hazırlık süreci; **AWS SDK v3** kütüphanelerinin kullanımı, paket yönetimi (**NPM**) ve sunucu tarafı çalışma ortamlarının yapılandırılması konularında derinlemesine tecrübe kazandırmıştır. Ayrıca, ham verinin bir veritabanı şemasına dönüştürülmesi ve bulut kaynaklarının terminal (CLI) üzerinden yönetilmesi gibi kritik mühendislik becerileri bu uygulama sayesinde pekiştirilmiştir.

- **Gelecek Perspektifi:** Elde edilen bu başarı, projenin bir sonraki aşamasında verilerin görselleştirilmesi (Dashboard) veya eşik değerler aşıldığında otomatik bildirimlerin gönderilmesi (SNS/Lambda) gibi daha gelişmiş bulut senaryolarına zemin hazırlamaktadır.

6. KAYNAKÇA

Raporun hazırlanması ve uygulamanın geliştirilmesi sürecinde aşağıdaki kaynaklardan yararlanılmıştır:

1. **AWS Documentation:** DynamoDB Developer Guide
2. **Node.js Official Documentation:** v18.x API Reference
3. **AWS SDK for JavaScript:** SDK v3 Client Library